

AI-gestuurd dependency dashboard voor automatische analyse en prioritering van software-updates.

Jelle Vandriessche.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Aelbrecht

Co-promotor: Dhr. B. Pattyn

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Deze bachelorproef ontwikkelt een aparte webapplicatie (dashboard) met een from-scratch ontworpen AI-agent om dependency-updates automatisch te detecteren, analyseren en prioriteren voor het kernteam dat platform- en dependencybeheer opneemt binnen een bedrijf. Vanuit de vastgestelde problematiek — veel manueel werk bij het interpreteren van changelogs, ophopende technische schuld en beperkte context bij updates — controleert de applicatie GitHub-repositories op gebruikte dependencies, haalt versie-informatie op uit publieke registries (zoals npm) en laat de AI-agent changelogs en release notes samenvatten met indicatie van criticiteit (security, bugfix, feature) en risico-impact.

Via een systematische aanpak met requirements-analyse, architectuurontwerp, proof-of-concept implementatie en gerichte vergelijking van AI-aanpakken wordt onderzocht hoe deze oplossing de manuele beoordelingslast kan verminderen en het overzicht op technische schuld kan verbeteren. De evaluatie met het kernteam focust op bruikbaarheid van het dashboard, duidelijkheid van AI-samenvattingen en ondersteuning bij prioritering. Verwacht wordt dat de aanpak een efficiënter, transparanter dependencybeheerproces oplevert dat herhaalbaar is en kan worden opgeschaald naar andere teams binnen de organisatie.

Inhoudsopgave

Lijst van figuren	viii
Lijst van tabellen	ix
Lijst van codefragmenten	x
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	3
1.4 Opzet van deze bachelorproef	4
2 Stand van zaken	6
2.1 Dependencybeheer en technische schuld	6
2.1.1 Dependency management en kwetsbare dependencies	6
2.1.2 Technische schuld en onderhoudslast	7
2.1.3 Dependencybots en hun beperkingen	7
2.2 Web- en datastack voor het AI-gestuurde dashboard	8
2.3 AI-agents en workflow-automatisering	9
2.3.1 AI-agents en orkestratie	9
2.4 Onderzoeksniche: AI-ondersteund dependencybeheer	12
3 Requirementsanalyse	13
3.1 Stakeholders	13
3.2 Functionele vereisten	14
3.2.1 Authenticatie en gebruikersbeheer	14
3.2.2 Projectbeheer	14
3.2.3 Dependency scanning en versiebeheer	15
3.2.4 AI-analyse en samenvatting	15
3.2.5 Dashboard en visualisatie	16
3.2.6 Notificaties en communicatie	16
3.2.7 Regels en beslissingsondersteuning	17
3.3 Niet-functionele vereisten	17
3.3.1 Beveiliging	17
3.3.2 Prestaties	17
3.3.3 Schaalbaarheid	18
3.3.4 Betrouwbaarheid en foutafhandeling	18

3.3.5	Onderhoudbaarheid	18
3.3.6	Bruikbaarheid	18
3.3.7	Traceerbaarheid en audit	19
3.3.8	Portabiliteit en installatie	19
3.4	Samenvattende MoSCoW-prioritering	19
4	Methodologie	20
4.1	Fase 1 - Verdiepende literatuurstudie en probleemverkenning	20
4.2	Fase 2 - Requirements-analyse en architectuurontwerp	21
4.3	Fase 3 - Implementatie van het proof-of-concept	21
4.4	Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak	22
4.5	Fase 5 - Evaluatie en rapportering	23
5	Ontwerp en implementatie van het AI-gestuurde dashboard	25
5.1	Architectuur van het systeem	25
5.2	Implementatie van de Mastra-agent	28
5.3	Implementatie van de n8n-workflow	32
5.4	Implementatie van de OpenAI-agent-builder	35
5.5	Integratie met de webapplicatie	35
6	Resultaten en evaluatie	36
7	Conclusie	37
A	Onderzoeksvoorstel	39
A.1	Inleiding	39
A.1.1	Probleemstelling	39
A.1.2	Onderzoeksvraag	40
A.1.3	Onderzoeksdoelstelling	41
A.2	Literatuurstudie	42
A.2.1	Dependency management en kwetsbare dependencies	42
A.2.2	Technische schuld en onderhoudslast	43
A.2.3	Dependencybots en hun beperkingen	43
A.2.4	AI, agents en workflow automatisering	43
A.2.5	Onderzoeksniche: AI-ondersteund dependencybeheer	44
A.3	Methodologie	45
A.3.1	Fase 1 - Verdiepende literatuurstudie en probleemverkenning	45
A.3.2	Fase 2 - Requirements-analyse en architectuurontwerp	45
A.3.3	Fase 3 - Implementatie van het proof-of-concept	46
A.3.4	Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak	46
A.3.5	Fase 5 - Evaluatie en rapportering	47
A.4	Verwacht resultaat en conclusie	48

Lijst van figuren

4.1 Fasering en tijdsplanning van het onderzoek	24
5.1 Architectuuroverzicht DepGuard AI	26
5.2 Conceptueel datamodel DepGuard AI	27
5.3 AI-workflow in n8n	35
A.1 Fasering en tijdsplanning van het onderzoek	49

Lijst van tabellen

Lijst van codefragmenten

5.1	Mastra dependencyAgent-configuratie met tools en research-strategie	29
5.2	fetchNpmInfoTool: npm-metadata en GitHub release notes met CHAN-GELOG fallback	31
5.3	webChangelogSearchTool: webzoekopdracht naar changelogs en migratieguides	32
5.4	Aanroep van de n8n-workflow	34

1

Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden Pashchenko et al. (2018). In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld Behutiye et al. (2024) en Pashchenko et al. (2018). Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt Pashchenko et al. (2018). stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als *technische schuld* (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn Behutiye et al. (2024) en Ruiz et al. (2024). Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit Besker

(2020). Om deze last te verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn, Erlenhov et al. (2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd Erlenhov et al. (2022). Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van Springbok. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol.

1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform- en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt. <https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt. <https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem. <https://www.npmjs.com>

2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van Springbok?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen Springbok?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van Springbok?
7. In welke mate presteren de geselecteerde AI-agents effectief bij het analyseren en prioriteren van dependency-updates, gemeten in termen van nauwkeurigheid, bruikbaarheid van de adviezen en vermindering van manueel werk voor het kernteam?

1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van Springbok voor geautomatiseerd dependencybeheer. De webapplicatie fungeert daarbij als losstaand platform naast GitHub en gebruikt GitHub voornamelijk als bron voor project- en dependency-informatie en als kanaal om eventuele vervolgacties (zoals pull requests of issues) te initiëren.

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registries (zoals npm);

- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van Springbok. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van Springbok;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen Springbok.

1.4. Opzet van deze bachelorproef

De bachelorproef volgt een klassieke onderzoeksstructuur en is als volgt opgebouwd:

- **Hoofdstuk 1** Dit hoofdstuk beschrijft de context, de probleemstelling, de onderzoeksvragen en de doelstellingen van de bachelorproef.
- **Hoofdstuk 2** In dit hoofdstuk wordt de relevante literatuur rond dependency management, bestaande tools en oplossingen, technische schuld en AI-agentframeworks en workflow-automatisering geanalyseerd.
- **Hoofdstuk 3** Dit hoofdstuk bevat de requirementsanalyse voor het AI-gestuurde dependencydashboard in de context van Springbok.
- **Hoofdstuk 4** Dit hoofdstuk bespreekt de onderzoeksaanpak, het ontwerp van de systeemarchitectuur, de selectie van platformen en de implementatie van de proof-of-concept.
- **Hoofdstuk 5** Dit hoofdstuk licht het ontwerp en de implementatie toe van het proof-of-concept, inclusief de Mastra-agent en de n8n-workflow.
- **Hoofdstuk 6** In dit hoofdstuk worden de resultaten van de implementatie en experimenten gepresenteerd, en wordt de oplossing geëvalueerd op basis van de gedefinieerde vereisten en criteria.

- **Hoofdstuk 7** Dit hoofdstuk vat de belangrijkste bevindingen samen en formuleert aanbevelingen voor verdere uitwerking en mogelijke toekomstige verbeteringen.

2

Stand van zaken

Dit hoofdstuk bespreekt de relevante stand van zaken rond dependency management, technische schuld, dependencybots en recente ontwikkelingen in AI-agents en workflow-automatisering. Het bouwt voort op de probleemstelling uit hoofdstuk 1 en biedt het theoretische kader dat nodig is om de onderzoeksvragen en de ontwerpkeuzes voor het voorgestelde AI-gestuurde dependencyplatform te onderbouwen. Na de bespreking van de literatuur wordt in hoofdstuk 4 uiteengezet hoe deze inzichten vertaald worden naar de concrete aanpak en implementatie van de proof-of-concept.

2.1. Dependencybeheer en technische schuld

2.1.1. Dependency management en kwetsbare dependencies

Moderne softwareprojecten steunen sterk op open-source dependencies, waardoor kwetsbaarheden in externe bibliotheken rechtstreeks een impact kunnen hebben op de veiligheid en betrouwbaarheid van toepassingen Pashchenko et al. (2018). tonen aan dat een groot deel van de aangetroffen kwetsbare dependencies in de praktijk relatief eenvoudig verholpen kan worden door te upgraden naar een veiligere versie, maar dat ontwikkelteams deze updates vaak niet consequent doorvoeren. Dat wijst niet alleen op een technisch probleem (kwetsbare libraries), maar ook op een organisatorische uitdaging: teams missen tijd en tooling om systematisch zicht te houden op alle dependency-updates Pashchenko et al. (2018).

In hun proefondervindelijk onderzoek onderscheiden Pashchenko et al. (2018) tussen daadwerkelijk gedeployde kwetsbare dependencies en libraries die enkel tijdens ontwikkeling of testing gebruikt worden, en stellen zij vast dat het merendeel van de reëel kwetsbare dependencies door de eigen ontwikkelaars of directe onderhouders kan worden opgelost. Dit onderstreept het belang van systematisch dependencybeheer en duidelijke inzichten in welke dependencies effectief risico vor-

men in productieomgevingen Pashchenko et al. (2018). Voor deze bachelorproef is dat een belangrijk uitgangspunt: een AI-gestuurd dashboard moet in de eerste plaats helpen om de relevante, risicoverhogende dependencies zichtbaar en beheersbaar te maken voor het kernteam.

2.1.2. Technische schuld en onderhoudslast

Het uitstellen van dependency-updates draagt bij aan de opbouw van technische schuld, waardoor onderhoud en toekomstige wijzigingen steeds meer inspanning vereisen Behutiye et al. (2024) en Ruiz et al. (2024). Behutiye et al. (2024) analyseren de rol van technische schuld in agile omgevingen en beschrijven hoe keuzes die op korte termijn tijdswinst opleveren, op langere termijn leiden tot hogere kosten en complexere wijzigingen. Ruiz et al. (2024) bespreken in hun tertiaire studie dat technischeschuldmanagement de voorbije jaren een groeiend onderzoeksdomein is geworden, waarbij organisaties worstelen met het in kaart brengen, prioriteren en terugdringen van opgebouwde schuld. Empirisch werk van Chalmers University of Technology bevestigt dat technische schuld niet louter een theoretisch concept is, maar in industriële context concreet voelbaar is in de vorm van verminderde productiviteit, hogere foutkans en vertragingen in softwarelevering Besker (2020). In de context van dependencybeheer betekent dit dat een gebrek aan inzicht en prioritering rond updates zich vertaalt in een groeiende onderhoudsachterstand, precies wat het voorgestelde AI-dashboard moet helpen vermijden door updates beter te signaleren, te duiden en te prioriteren.

2.1.3. Dependencybots en hun beperkingen

Hoewel dependencybots het detecteren en aanmaken van update-pull-requests automatiseren, blijft de inhoudelijke beoordeling van changelogs, compatibiliteit en risico's grotendeels bij ontwikkelaars liggen, wat aansluit bij de propleemstelling van deze bachelorproef Erlenhov et al. (2022) en Pashchenko et al. (2018). De literatuur suggereert dat bots vooral sterk zijn in het uitvoeren van repetitieve, goed afgebakende acties (zoals het openen van pull requests), maar minder in het leveren van context en uitleg bij de voorgestelde wijzigingen. Dit opent een duidelijke niche voor een aanvullende, AI-gestuurde laag die juist die interpretatie- en beslissingsondersteuning biedt.

In de praktijk botsen dependencybots zoals Dependabot bovendien op beperkingen rind private package-ecosystemen. Het is bijvoorbeeld moeilijk, en in sommige setups zelfs niet mogelijk, om Dependabot correct te configureren voor packages die uitsluitend via een private npm-registry op GitHub of een intern registry beschikbaar zijn. Daardoor blijven kritieke updates in private packages soms onder de radar, of moeten teams terugvallen op eigen scripts en manuele controles om private dependencies op te volgen.

Voor deze bachelorproef is ook de toegang tot private repositories en registries

een belangrijk aandachtspunt. GitHub biedt verschillende mogelijkheden om private repos en bijhorende package.json bestanden of registries te benaderen, onder meer via authenticatie met een persoonlijk access token of via OAuth/SSo met een Google- of GitHub-account. In de context van het beoogde dashboard zijn twee benaderingen relevant: ofwel logt een gebruiker in via een identity-provider (zoals Google) die aan zijn GitHub-account is gekoppeld en verleent hij daarmee de nodige rechten aan de applicatie, ofwel genereert de gebruiker die een project maakt een GitHub-personal-access-token en registreert hij dit token bij het nieuwe project. Beide opties maken het mogelijk om, binnen de toegestane rechten, ook informatie uit private repositories en private registries op te halen, iets waar standaard dependencybots in de praktijk niet altijd vlot in slagen.

2.2. Web- en datastack voor het AI-gestuurde dashboard

Het proof-of-concept wordt gerealiseerd als een moderne webapplicatie op basis van Node.js en Next.js, volledig getypeerd met TypeScript. Deze stack is bijzonder geschikt voor het bouwen van interactieve dashboards en API-gestuurde backends, en sluit aan bij recente richtlijnen voor schaalbare, type-veilige webtoepassingen met Next.js, Supabase en hedendaagse UI-bibliotheken. Next.js biedt ondersteuning voor server-side rendering, API-routes en een component-gebaseerde architectuur, wat het integreren van AI-functionaliteit en externe services (zoals GitHub en registries) vereenvoudigt.

Supabase wordt gebruikt als backend-dienst voor opslag en beheer van project-, dependency- en analysegegevens. Het combineert een PostgreSQL-database met een geïntegreerde API-laag en authenticatie, waardoor het mogelijk is om snel een betrouwbare gegevenslaag op te zetten voor het dashboard. Voor communicatie naar ontwikkelaars, zoals notificaties over belangrijke updates, wordt Resend ingezet als e-maildienst, terwijl Tailwind CSS en shadcn/ui zorgen voor een consistente en efficiënte uitwerking van de gebruikersinterface in React-componenten.

De AI-functionaliteit wordt gerealiseerd via externe taalmodellen, zoals die beschikbaar zijn via de OpenAI API of Google AI Studio. Deze modellen worden aangestuurd vanuit de TypeScript-code in Mastra (in het geval van de code-gedreven agent), vanuit n8n-workflows (in het geval van de low-code aanpak) of via een door OpenAI aangeboden agent builder waarmee agents en hun tools declaratief kunnen worden geconfigureerd. Door AI-aanroepen te combineren met de hierboven beschreven web- en datastack ontstaat een geïntegreerd platform waarin dependency-informatie centraal opgeslagen, geanalyseerd en gevisualiseerd kan worden.

2.3. AI-agents en workflow-automatisering

2.3.1. AI-agents en orkestratie

Recente literatuur verkent hoe AI-agents en multi-agentframeworks kunnen worden ingezet om complexe taken in softwareontwikkeling en data-intensieve workflows te coördineren Joshi (2025), Kim et al. (2024), en Zhu et al. (2025). Joshi biedt een overzicht van autonome systemen en collaboratieve AI-agentframeworks en benadrukt dat zulke agents vooral nuttig zijn in scenario's met veel herhalende beslissingen op basis van tekstuele en gestructureerde input, zoals logs, notificaties en rapporten Joshi (2025).

introduceren met Bel Esprit een multi-agentframework voor het opbouwen van AI-model-pijplijnen, waarbij verschillende gespecialiseerde agents samenwerken om complexere taken modulair af te handelen Kim et al. (2024). bestuderen in OA-agents empirisch welke ontwerpkeuzes leiden tot effectieve agents, en bespreken onder meer het belang van duidelijke taakafbakening, robuuste toolintegratie en feedbackloops voor betrouwbare beslissingsondersteuning Zhu et al. (2025). Deze inzichten zijn relevant voor het ontwerp van een AI-agent die dependency-updates interpreteert, samenvat en beslissingsvoorstellen formuleert voor een onderhoudsteam.

Naast generieke agentframeworks beschrijft Wali et al. hoe workflow-automatisering met n8n¹ operationele efficiëntie kan verhogen door repetitieve, gegevensgedreven processen te coördineren over verschillende systemen heen Wali et al. (2025). Hoewel hun casus zich richt op een financiële context, illustreert de studie dat low-code workflowtools geschikt zijn om integraties met externe APIs, databronnen en notificatiekanalen te realiseren, wat ook essentieel is voor een geautomatiseerde dependency-updatepipeline Wali et al. (2025).

Recente studies tonen bovendien aan dat AI-technieken succesvol kunnen worden ingezet in onderhoudstaken zoals changelog-analyse, code review en beslissingsondersteuning Behutiye et al. (2024), Joshi (2025), Kim et al. (2024), en Zhu et al. (2025). Daarmee wordt de haalbaarheid en relevantie van een AI-gestuurde benadering voor dependencybeheer onderbouwd: veel van de informatie die ontwikkelaars vandaag manueel verwerken (release notes, changelogs, pull-requestbeschrijvingen) bestaat uit tekst en gestructureerde metadata, wat precies het type input is waarop AI-agents goed kunnen presteren.

Mastra: TypeScript-framework voor AI-agents

Naast algemene concepten en framework-onafhankelijke benaderingen voor AI-agents zijn er de laatste jaren specifieke ontwikkelframeworks ontstaan die het bouwen, testen en monitoren van agents vereenvoudigen. Een recent voorbeeld is

¹n8n is een open-source tool om workflows en API's visueel te automatiseren. <https://n8n.io>

Mastra², een open-source framework uit het JavaScript- en TypeScript-ecosysteem voor het bouwen van AI-gestuurde agents en workflows AI (2026). Mastra is ontworpen rond gangbare patronen voor agentie en evaluaties, met integratie in bestaande webstacks zoals Node.js en Next.js AI (2025a, 2026).

In Mastra wordt een *agent* gedefinieerd als een combinatie van een taalmodel, instructies en optioneel geheugen en tools. Tools zijn gewone functies die door de agent kunnen worden aangeroepen om extra context of data op te halen (bijvoorbeeld externe APIs of interne services), waardoor complexe taken modulair kunnen worden opgebouwd Nwachukwu (2025). Daarnaast biedt het framework ondersteuning voor workflows waarin meerdere stappen, tool-calls en beslissingslogica gecombineerd worden, wat nuttig is voor scenario's zoals het sequentieel verwerken van dependency-informatie en changelogs The AI Builders (2025).

Een bijkomend aandachtspunt in Mastra is observability: het framework voorziet tracing en logging van modelinteracties, agentbeslissingen, tool-calls en workflowstappen, zodat ontwikkelaars kunnen analyseren hoe een agent tot een bepaalde output is gekomen en eventuele fouten of inefficiënties kunnen opsporen AI (2025b). Dit sluit aan bij de nood aan transparantie en debugbaarheid in AI-ondersteunde onderhoudsprocessen, zoals bij het automatisch interpreteren van changelogs en dependency-updates.

In het kader van deze bachelorproef wordt Mastra gebruikt om een eerste AI-integratie te realiseren: een agent die via een zelf ontwikkelde tool changelog- en dependency-informatie kan ophalen, analyseren en samenvatten. Deze tool-ondersteunde agent vormt een concrete technische invulling van de in de literatuur besproken agent-concepten en dient als basis om te onderzoeken in welke mate een dergelijke benadering effectief ondersteuning biedt bij dependencybeheer in een bedrijfscontext. Mastra sluit goed aan bij het technologische ecosysteem van deze bachelorproef, dat gebaseerd is op Node.js, Next.js en TypeScript. Omdat Mastra zelf een TypeScript-framework is, kan de AI-agent rechtstreeks geïntegreerd worden in dezelfde codebase als de webapplicatie, zonder extra taal- of runtimegrenzen. Dit vereenvoudigt het hergebruiken van bestaande datamodellen, services en API-clients (zoals GitHub- en registry-clients) binnen de agenttools. Bovendien ondersteunt Mastra het definiëren van tools die externe APIs, zoals de GitHub API, kunnen aanroepen, wat essentieel is om automatisch informatie over repositories, releases en changelogbestanden op te halen.

n8n: low-code workflow- en AI-automatisering

Naast code gerichte agentframeworks zoals Mastra zijn er ook low-codeplarichten op het visueel modelleren van workflows en integraties. Een belangrijk voorbeeld is n8n (uitgesproken als “n-eight-n”), een open-source, event-gedreven workflow-automatiseringstool die een visuele, node-gebaseerde editor aanbiedt om stap-

²Mastra is een AI-native framework voor het bouwen en orkestreren van AI-agents en workflows.<https://mastra.ai>

pen in een proces met elkaar te verbinden n8n GmbH (2025a, 2025b). n8n wordt gepositioneerd als een hybride no-code/low-code oplossing: eenvoudige automatiseringen kunnen volledig via de grafische interface worden opgebouwd, terwijl Function-nodes en eigen nodes toelaten om JavaScript-code en maatwerklogica te integreren waar nodig Proser (2025).

n8n ondersteunt honderden kant-enklare integraties (nodes) met externe diensten, waaronder Github, databases, e-mailproviders en verschillende AI- en LLM-aanbieders Codecademy (2025) en Hatchworks (2025). Workflows kunnen worden getriggerd door uiteelopenende events, zoals webhooks, geplande cron-jobs, inkomende berichten of formulierinzendingen, en kunnen data stap voor stap transformeren, verrijken en routeren naar andere systemen Infralovers (2025) en Proser (2025). Dankzij deze event-gedreven architectuur is n8n geschikt voor het orkestreren van AI-pijplijnen waarin data uit meerdere bronnen wordt samengebracht, verwerkt en doorgegeven aan taalmodellen of agents.

Recent heeft n8n zijn focus expliciet uitgebreid naar AI-workflows en AI-agents: het platform biedt gespecialiseerde nodes voor LLM-providers (zoals OpenAI, Google Gemini en andere modellen) en functies voor promptconstructie, contextverrijking en post-processing Codecademy (2025) en Hatchworks (2025). Daardoor kunnen ontwikkelaars end-to-end AI-workflows opzetten, bijvoorbeeld om inkomende gegevens te analyseren met een taalmodel en op basis van de uitkomst vervolgacties te triggeren, zonder de visuele editor te verlaten Hatchworks (2025).

In het kader van deze bachelorproef wordt n8n gebruikt om een tweede AI-integratie te realiseren in de vorm van een workflow. Deze workflow automatiseert de orkestratie van dependency-gerelateerde taken: het ophalen van dependency- of versie-informatie, het aanroepen van een taalmodel voor analyse of samenvatting, en het terugsturen van de resultaten naar de webapplicatie of een ander kanaal. n8n fungeert hierbij als een visuele orchestrator rond de AI-component, wat een concreet contrast biedt met de meer code-gedreven aanpak met Mastra en inzicht geeft in de voor- en nadelen van low-code AI workflows in de context van dependencybeheer.

Voor dependencybeheer is vooral de integratie tussen n8n en GitHub relevant: n8n biedt kant-en-klare nodes om repositorygegevens op te halen, issues en pull requests te beheren en notificaties te versturen bij nieuwe releases of commits. Daardoor kan een workflow bijvoorbeeld automatisch getriggerd worden wanneer een nieuwe release in een GitHub-repository verschijnt, waarna de relevante metadata en changelog-informatie naar een taalmodel of agent gestuurd wordt ter analyse. In deze bachelorproef wordt deze mogelijkheid benut om een alternatieve, low-code orkestratie van dependency-updates uit te werken, in contrast met de meer code-gedreven integratie met Mastra.

2.4. Onderzoeksniche: AI-ondersteund dependencybeheer

Samengenomen schetst de literatuur een duidelijk beeld: kwetsbare of verouderde dependencies vormen een reëel risico, maar kunnen vaak verholpen worden door updates die vandaag onvoldoende systematisch worden uitgevoerd Behutiye et al. (2024) en Pashchenko et al. (2018). Tegelijk tonen studies naar technische schuld dat het structureel uitstellen van onderhoud, waaronder dependency updates, leidt tot een groeiende onderhoudslast en achterstand Behutiye et al. (2024), Besker (2020), en Ruiz et al. (2024).

Onderzoek naar dependencybots maakt duidelijk dat huidige oplossingen weliswaar updates detecteren en pull requests genereren, maar ontwikkelaars nog steeds belasten met het interpreteren van changelogs en het beoordelen van risico's en prioriteiten Erlenhov et al. (2022). De recente vooruitgang in AI-agents en workflow-automatisering suggereert dat een volgende stap mogelijk is: een systeem waarin een AI-agent niet alleen updates signaleert, maar ook de inhoud van release notes en changelogs analyseert, samenvat en contextafhankelijk advies geeft aan een klein kernteam dat verantwoordelijk is voor dependencybeheer Joshi (2025), Kim et al. (2024), Wali et al. (2025), en Zhu et al. (2025).

Deze bachelorproef positioneert zich precies in deze niche door een AI-gestuurde laag bovenop bestaande dependencybots te onderzoeken en te prototypen, met de expliciete focus op het verlagen van de manuele beoordelingslast en het beheersbaar houden van technische schuld rond dependencies in een bedrijfscontext.

3

Requirementsanalyse

Deze requirementsanalyse beschrijft de functionele en niet-functionele vereisten voor DepGuard AI, een AI-gestuurd dependencydashboard ontwikkeld als bachelor-proef ontwerp en de implementatie van het proof-of-concept en bouwt voort op de probleemstelling, literatuurstudie en overleg met de co-promotor. Het platform wordt opgezet als een losstaande webapplicatie, onafhankelijk van bestaande tools zoals Dependabot, en richt zich op het samenvatten van release notes, het beoordelen van de criticiteit van updates en het ondersteunen van het kernteam bij het prioriteren van dependency-updates.

3.1. Stakeholders

De belangrijkste stakeholders van DepGuard AI zijn:

- **Kernteam voor platform- en dependencybeheer:** primaire gebruikers, verantwoordelijk voor het opvolgen en doorvoeren van dependency-updates en dagelijks gebruik van het dashboard voor overzicht, prioritering en AI samenvattingen.
- **Developers binnen projecten:** secundaire gebruikers die aan individuele projecten werken, update-adviezen raadplegen en projectleden en dependencies beheren.
- **Co-promotor / bedrijfsbegeleider:** opdrachtgever vanuit Springbok, die de bruikbaarheid en kwaliteit van de oplossing evalueert en mee de requirements scherpstelt.
- **Promotor (HoGent):** academische begeleider die de methodologie, uitvoering en kwaliteit van het eindwerk beoordeelt.

- **Externe platforms:** GitHub en de npm-registry leveren project- en versiemetadata via hun APIs en vormen zo cruciale bronnen voor het systeem.

3.2. Functionele vereisten

De functionele vereisten zijn gegroepeerd per domein en geprioriteerd volgens de MoSCoW-methode (Must Have, Should Have, Won't Have voor dit proof-of-concept).

3.2.1. Authenticatie en gebruikersbeheer

De volgende functionele vereisten gelden voor authenticatie en gebruikersbeheer.

- **F-01 (Must Have):** Gebruikersregistratie en login via e-mail en wachtwoord (Supabase Auth).
- **F-02 (Must Have):** Aanmaken van een bedrijfsaccount (multi-tenant).
- **F-03 (Must Have):** Uitnodigen van teamleden via e-mail met rolkoppeling (Resend API).
- **F-04 (Must Have):** Rollen per bedrijf: owner, project_lead, developer, tester (RBAC).
- **F-05 (Must Have):** Rollen per project: project_lead, developer (projectniveau RBAC).
- **F-06 (Should Have):** Uitnodiging accepteren via directe link zonder e-mail (fallbackflow).
- **F-07 (Could Have):** SSO / OAuth login (bijvoorbeeld GitHub of Google via Supabase OAuth).

3.2.2. Projectbeheer

De volgende functionele vereisten gelden voor projectbeheer.

- **F-08 (Must Have):** Aanmaken, bewerken en verwijderen van projecten (CRUD).
- **F-09 (Must Have):** Koppelen van een publieke GitHub-repository aan een project (GitHub-integratie).
- **F-10 (Must Have):** Uploaden van een package.json om dependencies in te laden.
- **F-11 (Must Have):** Toewijzen van teamleden aan projecten met projectrol.
- **F-12 (Must Have):** Overzicht van alle projecten per bedrijf met statusindicator.
- **F-13 (Could Have):** Archiveren of deactiveren van projecten.

3.2.3. Dependency scanning en versiebeheer

De volgende functionele vereisten gelden voor dependency scanning en versiebeheer.

- **F-14 (Must Have):** Automatisch detecteren van dependencies via GitHub (uitlezen van `package.json` via de GitHub API).
- **F-15 (Must Have):** Ophalen van de meest recente versie via de npm-registry (`npmjs.org`).
- **F-16 (Must Have):** Vergelijken van huidige versie met meest recente versie (semver-vergelijking).
- **F-17 (Must Have):** Classificeren van updates als major, minor of patch (semver-classificatie).
- **F-18 (Must Have):** Opslaan van dependency-informatie (naam, versies, status) in Supabase Postgres.
- **F-19 (Should Have):** Manueel hertriggeren van een scan door de gebruiker.
- **F-20 (Could Have):** Automatisch periodiek scannen via een geplande job (cron).
- **F-21 (Won't Have):** Ondersteuning voor meerdere registries (bijv. PyPI, Maven).

3.2.4. AI-analyse en samenvatting

De volgende functionele vereisten gelden voor AI-analyse en samenvatting.

- **F-22 (Must Have):** AI-agent analyseert changelogs en release notes van npm en GitHub (Mastra-agent met LLM-backend).
- **F-23 (Must Have):** Genereren van een begrijpelijke samenvatting per dependency-update.
- **F-24 (Must Have):** Classificeren van het updatetype: security fix, bugfix, nieuwe feature, breaking change.
- **F-25 (Must Have):** Aanduiden of een update veilig, risicovol of kritiek lijkt (risico-indicatie).
- **F-26 (Must Have):** Opslaan van de AI-samenvatting per dependency in het veld `ai_summary`.
- **F-27 (Must Have):** Ondersteuning voor meerdere AI-providers (OpenAI, Google Gemini).

- **F-28 (Should Have):** Alternatieve AI-workflow via n8n (low-code orkestratie als vergelijking met de Mastra-agent).
- **F-28b (Should Have):** Alternatieve AI-agent via OpenAI Agent Builder, waarbij een gehoste agent met tools wordt geconfigureerd als tweede alternatief naast Mastra en n8n.
- **F-29 (Should Have):** Verrijken van AI-input met webcontext via Tavily.
- **F-30 (Should Have):** Hertriggere van AI-analyse op verzoek van de gebruiker.
- **F-31 (Could Have):** Gestructureerde output met migratiestappen en deprecation-warnings.

3.2.5. Dashboard en visualisatie

De volgende functionele vereisten gelden voor het dashboard en visualisatie.

- **F-32 (Must Have):** Overzichtsdashboard met statistieken (verouderde packages, security-issues, recente activiteit).
- **F-33 (Must Have):** Detailweergave per project met lijst van dependencies en hun status.
- **F-34 (Must Have):** Weergave van de AI-samenvatting per dependency in het dashboard.
- **F-35 (Must Have):** Visuele badge of kleurcodering voor major/minor/patch en risico-niveau.
- **F-36 (Should Have):** Filteren en sorteren van dependencies op status, updatetype en criticiteit.
- **F-37 (Could Have):** Exporteren van een dependency-rapport als PDF of CSV.

3.2.6. Notificaties en communicatie

De volgende functionele vereisten gelden voor notificaties en communicatie.

- **F-38 (Must Have):** E-mailnotificatie bij uitnodiging van een nieuw teamlid (Resend API).
- **F-39 (Should Have):** E-mailnotificatie bij detectie van een kritieke security-update.
- **F-40 (Could Have):** In-app notificatie of badge bij nieuwe updates per project.
- **F-41 (Won't Have):** Integratie met Slack of Microsoft Teams voor updateberichten.

3.2.7. Regels en beslissingsondersteuning

De volgende functionele vereisten gelden voor regels en beslissingsondersteuning.

- **F-42 (Must Have):** AI geeft een advies per dependency: “veilig te updaten”, “voorzichtigheid geboden” of “kritiek”.
- **F-43 (Should Have):** Mogelijkheid om een dependency handmatig als “genegeerd” of “gedaan” te markeren.
- **F-44 (Could Have):** Definieren van updatebeleid per project (bijvoorbeeld patch-updates automatisch goedkeuren).
- **F-45 (Won’t Have):** Automatisch aanmaken van pull requests of GitHub-issues op basis van advies.

3.3. Niet-functionele vereisten

De niet-functionele vereisten zijn afgestemd op de gekozen technische stack (Next.js, Supabase, Mastra, n8n, OpenAI-agent builder en externe LLms) en de context van Springbok.

3.3.1. Beveiliging

- **NFR-01:** Row Level Security (RLS) in Supabase zorgt ervoor dat gebruikers uitsluitend data zien van het eigen bedrijf en de toegewezen projecten.
- **NFR-02:** Authenticatie is verplicht voor alle API-routes en dashboardpagina’s via Next.js-middleware.
- **NFR-03:** API-sleutels (OpenAI, Google, Resend, GitHub) worden uitsluitend als omgevingsvariabelen opgeslagen en zijn nooit zichtbaar in de frontend.
- **NFR-04:** Uithodigingstokens zijn eenmalig geldig en vervallen na acceptatie.

3.3.2. Prestaties

- **NFR-05:** De end-to-end AI-analyse (van dependency-input tot opgeslagen samenvatting) wordt onder normale omstandigheden binnen ongeveer 30 seconden afgerond.
- **NFR-06:** Paginalaadtijden voor het dashboard blijven onder 2 seconden bij een dataset tot ongeveer 200 dependencies per bedrijf.
- **NFR-07:** De n8n-workflow en de Mastra-agent worden parallel geëvalueerd op doorlooptijd; de resultaten worden vastgelegd in de evaluatiefase.

3.3.3. Schaalbaarheid

- **NFR-08:** De multi-tenant architectuur laat toe om meerdere bedrijven, projecten en gebruikers onafhankelijk van elkaar te beheren.
- **NFR-09:** Opslag op Supabase (PostgreSQL) ondersteunt schaalbaarheid via managed infrastructuur en connection pooling.
- **NFR-10:** De AI-agentlaag is modulair en kan worden uitgebreid met extra tools, registries of LLM-providers zonder ingrijpende wijzigingen aan de webapplicatie.

3.3.4. Betrouwbaarheid en foutafhandeling

- **NFR-11:** Bij het ontbreken van release notes in GitHub wordt teruggevallen op bestanden zoals `CHANGELOG.md`, `CHANGES.md` of `HISTORY.md`.
- **NFR-12:** Als er geen changelog-informatie beschikbaar is, vermeldt de AI-agent dit expliciet in de samenvatting.
- **NFR-13:** API-fouten (GitHub, npm, LLM) worden gelogd en weergegeven als leesbare foutmeldingen in de UI.
- **NFR-14:** De Mastra-server en de Next.js-applicatie zijn onafhankelijk startbaar; bij uitval van Mastra blijft het dashboard bruikbaar zonder AI-analyse.

3.3.5. Onderhoudbaarheid

- **NFR-15:** De volledige codebase is getypeerd in TypeScript.
- **NFR-16:** Logica is opgesplitst per verantwoordelijkheid: AI-agentcode, API-routes en UI-componenten zijn in aparte modules gestructureerd.

3.3.6. Bruikbaarheid

- **NFR-17:** Het dashboard is responsive en bruikbaar op verschillende schermformaten.
- **NFR-18:** AI-samenvattingen zijn geschreven in begrijpelijke taal, gericht op developers zonder diepgaande AI-kennis.
- **NFR-19:** Statusbadges voor updatetype en risiconiveau zijn kleurgecodeerd en direct interpreteerbaar.
- **NFR-20:** Het onboardingproces (registratie, bedrijf aanmaken, project koppelen en eerste scan) kan in minder dan vijf minuten worden doorlopen.

3.3.7. Traceerbaarheid en audit

- **NFR-21:** Mastra voorziet tracing en logging van agentbeslissingen, tool-calls en modelinteracties, zodat het gedrag van de AI-agent kan worden geanalyseerd.
- **NFR-22:** Alle AI-samenvattingen worden met tijdstempel opgeslagen zodat de evolutie van adviezen per dependency navolgbaar is.
- **NFR-23:** Gebruikersacties zoals scannen, analyseren en uitnodigen zijn via Supabase auditeerbaar.

3.3.8. Portabiliteit en installatie

- **NFR-24:** De applicatie kan lokaal worden gestart via `npm run dev` (Next.js) en `npm run dev:mastra` (Mastra) na configuratie van de omgevingsvariabelen.
- **NFR-25:** De database-opzet is volledig scriptbaar en vereist geen manuele tabellaanmaak.
- **NFR-26:** De applicatie is inzetbaar op courante cloudplatformen, bijvoorbeeld Vercel voor Next.js en Supabase Cloud voor de databank.

3.4. Samenvattende MoSCoW-prioritering

In totaal worden 26 Must Have-, 9 Should Have-, 8 Could Have- en 4 Won't Have-vereisten onderscheiden. De Must Have-vereisten dekken de kernfunctionaliteit van DepGuard Ai (authenticatie, projectbeheer, scanning, AI-analyse, dashboard en basisadvies), terwijl de overige categorieën extra bruikbaarheid, gemak of toekomstige uitbreidingen beschrijven.

4

Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

4.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van Springbok geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

4.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kritici- teit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (logging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ontworpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De architectuur beschrijft onder meer de globale componenten (frontend, backend, databank, integraties met GitHub en registries), de verantwoordelijkheden per component, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data-modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de requirements.

4.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een aparte webapplicatie (dashboard) waarin per project de gebruikte dependencies worden bijgehouden en verrijkt met informatie uit publieke registries zoals npm. De applicatie werkt vanaf het begin met één of meerdere reële GitHub-repositories van Springbok, zodat de proof-of-concept onmiddellijk getest wordt in een realistische context. Dependency-informatie wordt automatisch uit deze repositories gehaald (bijvoorbeeld via repositoryconfiguratiebestanden) en opgeslagen in een centrale databank.

Parallel wordt de AI-agentlaag geïmplementeerd in de backend. In deze fase worden drie concrete AI-agentoplossingen opgezet en geïntegreerd in hetzelfde dashboard: een agent op basis van OpenAI Agent Builder, een workflow met n8n en een agentconfiguratie met Mastra AI. Elk van deze varianten ontvangt changelog- en release-note-informatie, genereert beknopte samenvattingen en beoordeelt de

aard en vermoedelijke impact van updates (bijvoorbeeld beveiligingsfix, bugfix, nieuwe functionaliteit, potentieel brekende wijziging). De frontend visualiseert de status van dependencies, aanbevolen acties en de door de verschillende AI-varianten gegenereerde toelichting. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar GitHub en één dependency-registries;
- drie geïmplementeerde AI-agentvarianten (OpenAI Agent Builder, n8n, Mastra AI) die changelogs en release notes verwerken en samenvatten;
- technische documentatie over de implementatie (architectuur, API's, datamodellen en integratiepunten voor de agents).

4.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requirements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);
- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Mastra AI onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeteste varianten;

- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

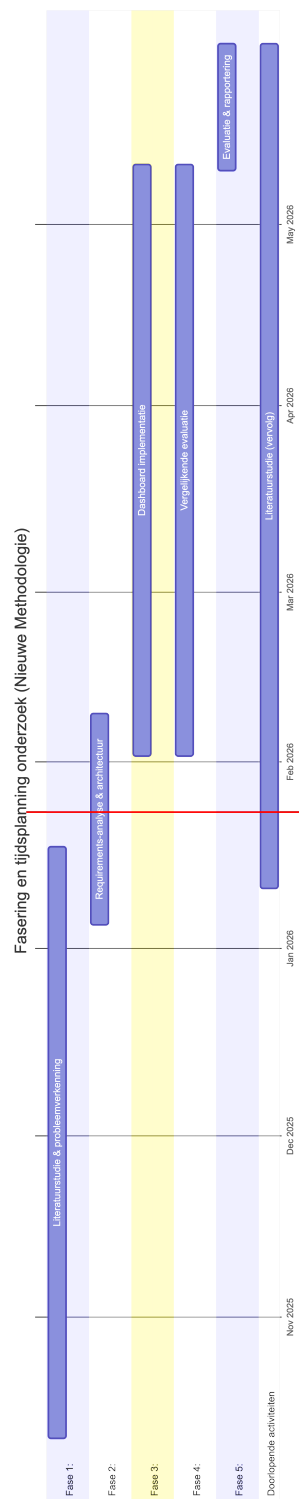
4.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd met de co-promotor binnen Springbok, aan de hand van een combinatie van demonstraties, gericht feedbackgesprekken en, waar mogelijk, beperkte praktijktesten realistische projecten. De evaluatie focust op bruikbaarheid van het dashboard, de gemeten vermindering van manuele analysetijd (bijvoorbeeld aantal geanalyseerde updates per uur vóór en na gebruik), de door de co-promotor toegekende beoordelingsscores voor duidelijkheid van de AI-samenvattingen en de mate waarin het systeem helpt bij de prioritering van updates.

De bevindingen uit deze evaluatie worden gecombineerd met de kwantitatieve resultaten van fase 4 om de onderzoeksvragen te beantwoorden en de onderzoekshypothesen te toetsen. Deliverables van fase 5 zijn:

- een synthese van de evaluatie met de co-promotor (inclusief gemeten tijds-winst, kwaliteitsbeoordelingen en verbetervoorstellen);
- de uitgewerkte bachelorproef met een geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;
- aanbevelingen voor verdere ontwikkeling en mogelijke integratie van het systeem in de bredere ontwikkelprocessen van Springbok.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur [A.1](#).



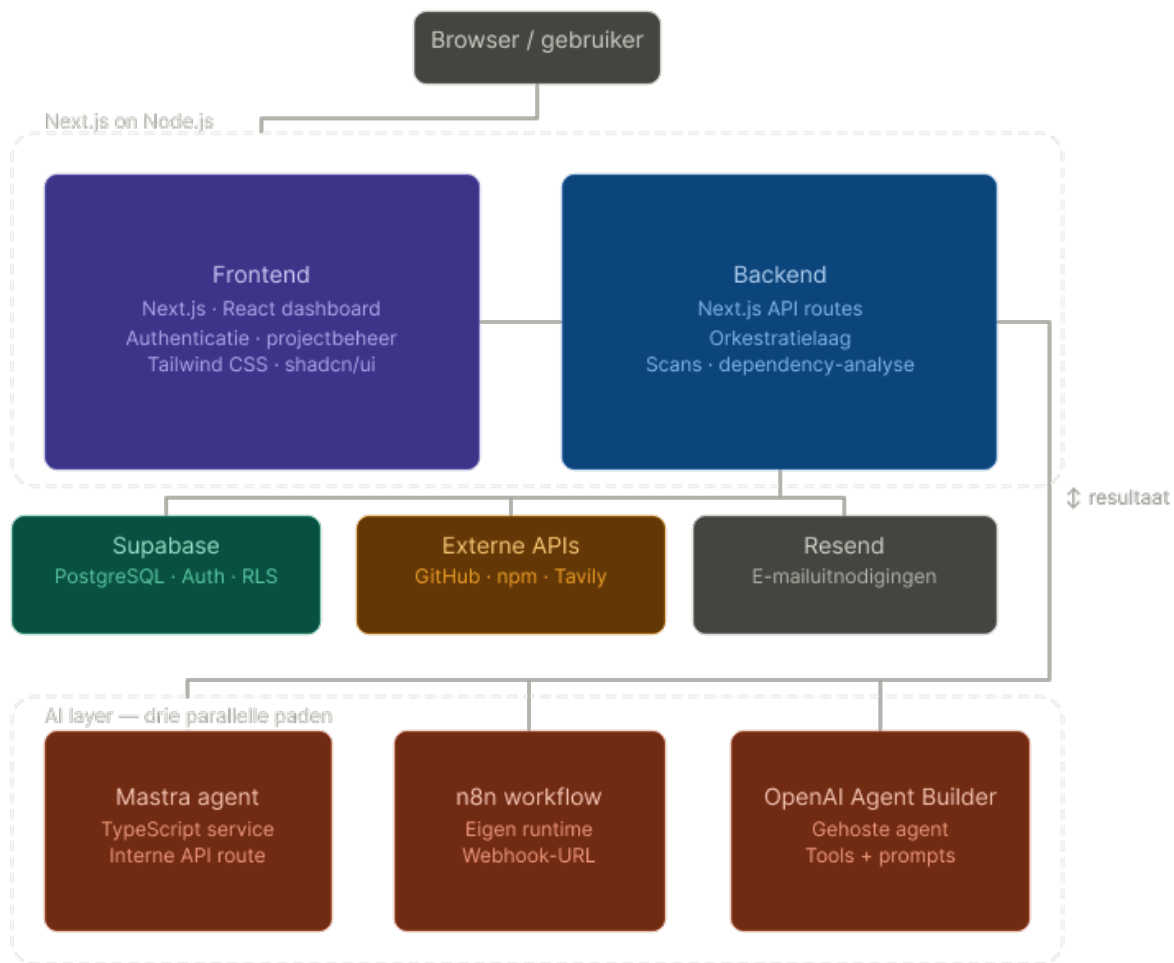
Figuur 4.1: Fasering en tijdsplanning van het onderzoek

5

Ontwerp en implementatie van het AI-gestuurde dashboard

5.1. Architectuur van het systeem

Het proof-of-concept voor DepGuard AI is opgezet als een moderne webarchitectuur met een duidelijke scheiding tussen frontend, backend, databank en AI-laag. De oplossing wordt gehost als een Next.js-applicatie bovenop Node.js, met Supabase als beheerde PostgreSQL-databank en Mastra, n8n en de OpenAI-agent-builder als afzonderlijke AI- en workflowcomponenten. Figuur 5.1 toont de globale systeemarchitectuur.



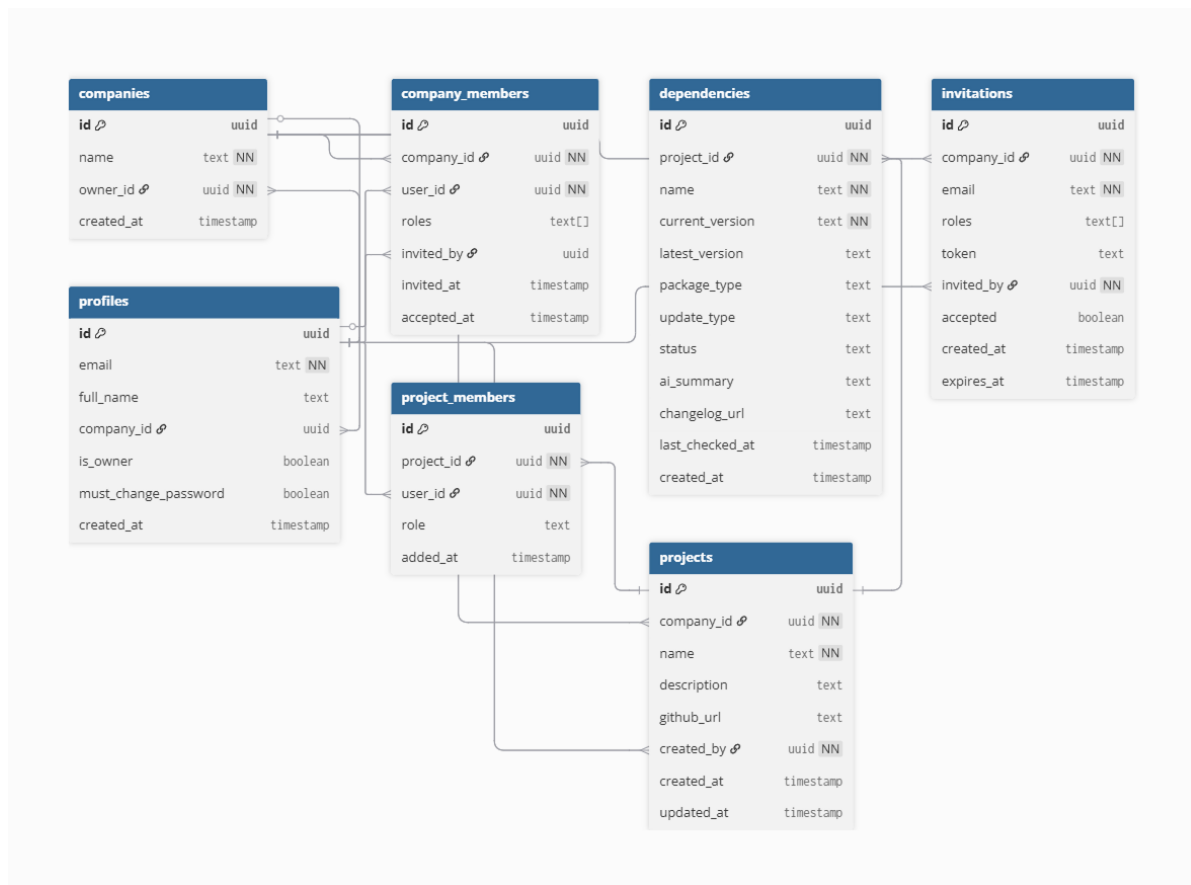
Figuur 5.1: Globale architectuur van DepGuard AI met frontend, backend, databank en AI-laag

De frontend is een Next.js-Reactdashboard dat authenticatie, projectbeheer en visualisatie van dependency- en AI-informatie verzorgt. Gebruikers kunnen bedrijven en projecten aanmaken, GitHub-repositories koppelen, dependency-scans opstarten en de door de AI gegenereerde samenvattingen en adviezen raadplegen. Voor de UI wordt gebruikgemaakt van Tailwind CSS en shadcn/ui, zodat componenten consistent en herbruikbaar zijn.

De backend bestaat uit een set Next.js API-routes die fungeren als orkestratielaag tussen de frontend, Supabase, externe APIs (GitHub, npm, Tavily) en de verschillende AI-componenten. Bij een scan leest de backend eerst de dependencyinformatie van een project (bijvoorbeeld uit `package.json` in GitHub), slaat de resultaten op in Supabase en roept vervolgens één van de AI-varianten aan: de mastra-agent, de n8n-workflow of de OpenAI-agent. De teruggestuurde samenvatting wordt gekoppeld aan de dependency en via het dashboard zichtbaar gemaakt voor de gebruiker.

Supabase verzorgt de persistente opslag van bedrijven, bedrijfleden, gebruikers, uitnodigingen, projecten, projectleden en dependencies, met Row Level Security

om multi-tenant isolatie te garanderen. Het conceptuele datamodel omvat onder meer de entiteiten `companies`, `company_members`, `profiles`, `invitations`, `projects`, `project_members` en `dependencies`. Figuur 5.2 toont het ERD met de belangrijkste relaties tussen deze entiteiten.



Figuur 5.2: Conceptueel ERD met bedrijven, projecten, leden en dependencies

De AI-laag bestaat uit drie parallelle paden die in aparte secties verder worden uitgewerkt. De Mastra-agent draait als TypeScript-service naast de Next.js-app en wordt aangeroepen via een interne API-route. De n8n-workflow draait in een eigen runtime en wordt aangesproken via een webhook-URL. De OpenAI-agent-builder wordt gebruikt om een gehoste agent te configureren met vergelijkbare tools en prompt. In alle gevallen verloopt de dataflow volgens hetzelfde patroon: dependencygegevens en eventueel changelog- en webcontext worden doorgestuurd naar de gekozen AI-component, de gestructureerde samenvatting komt terug naar de backend en wordt opgeslagen in Supabase. Hierdoor kan het dashboard de resultaten van de verschillende AI-aanpakken op een uniforme manier presenteren en vergelijken.

5.2. Implementatie van de Mastra-agent

Voor de eerste AI-integratie wordt gebruikgemaakt van Mastra, een TypeScript framework om AI-agents, tools en workflows te definiëren en te orchestreren. In de proof-of-concept wordt een `dependencyAgent` opgezet die specifiek is afgestemd op het analyseren van npm-dependency-updates. Deze agent combineert een taalmodel (bijvoorbeeld Google's `gemini-2.5-flash` of OpenAI's `gpt-4o-mini`) met twee eigen tools: `fetchNpmInfoTool` en `webChangelogSearchTool`.

De agent volgt een gestructureerde research-strategie: eerst haalt de `fetchNpmInfoTool` metadata en releasenotes op uit npm en GitHub; als die incompleet zijn, volgt de `webChangelogSearchTool` met een gerichte zoekopdracht naar changelogs. Het resultaat wordt gecombineerd en volgens een strikt JSON-formaat geformatteerd, zodat ontwikkelaars direct actionable inzichten krijgen over features, breaking changes, security fixes en migratiestappen. Listing 5.1 toont de definitie van de agent.

```

1 import { Agent } from "@mastra/core/agent";
2 import { google } from "@ai-sdk/google";
3 import { fetchNpmInfoTool } from "../tools/npm-tool";
4 import { webChangelogSearchTool } from
    "../tools/web-changelog-search-tool";
5
6 const MODEL = google("gemini-2.5-flash");
7
8 export const dependencyAgent = new Agent({
9   id: "DependencyAnalyzer",
10  name: "Dependency Analyzer",
11  instructions: `
12    You are a dependency update analyst for software developers.
13    Your goal is to give developers CONCRETE, SPECIFIC information about
14    package
15    updates so they do NOT have to read the full changelog themselves.
16
17    ## Research Strategy (follow IN ORDER)
18
19    ### Step 1 – fetch-npm-info
20    Always start by calling the fetch-npm-info tool with:
21    - packageName, fromVersion, toVersion
22
23    ### Step 2 – web-changelog-search
24    Call web-changelog-search when npm tool returns "No release notes
25    found".
26
27    ## Output Format
28    ## [package-name] v[from] → v[to] ([major/minor/patch] update)
29
30    > Source: [tool source or URL]
31
32    ### New Features      ### Breaking Changes
33    ### Security Fixes    ### Deprecated
34    ### Migration Steps   ### Safe to update?
35  `,
36  model: MODEL,
37  tools: {
38    fetchNpmInfoTool,
39    webChangelogSearchTool,
40  },
41 });

```

Codefragment 5.1: Mastra dependencyAgent-configuratie met tools en research-strategie

De `fetchNpmInfoTool` (Listing 5.2) is de primaire tool voor het ophalen van npm-

metadata en GitHub-release notes. De tool zoekt eerst naar een exacte release-tag (bijvoorbeeld v1.2.3), valt terug op de laatste vijf releases als die niet bestaat, en probeert vervolgens CHANGELOG-bestanden (CHANGELOG.md, HISTORY.md, ...) uit de repository te lezen. Dankzij fallback-logica en robuuste foutafhandeling levert de tool altijd een gestructureerd resultaat, zelfs als er geen officiële release notes zijn.

```

export const fetchNpmInfoTool = createTool({
  id: "fetch-npm-info",
  inputSchema: z.object({
    packageName: z.string(),
    fromVersion: z.string(),
    toVersion: z.string(),
  }),
  execute: async ({ packageName, fromVersion, toVersion }) => {
    // 1. npm registry metadata
    const npmData = await
      fetch(`https://registry.npmjs.org/${packageName}`).then(r =>
        r.json());

    // 2. GitHub releases (exact tag, dan recentste, dan CHANGELOG.md)
    const repoMatch = npmData.repository?.url?.match(/github\.com[/:](\[w.-\]
      -]+\[/[w.-]+\)/);
    if (repoMatch) {
      const [tagRes] = await Promise.allSettled([
        // Try exact tag variants
        ...["v${toVersion}", toVersion].map(tag =>
          fetch(`https://api.github.com/repos/${repoMatch[1]}/releases/tag,
            s/${tag}`)
            .then(r => r.json())
        )
      ]);
      // ... fallback naar CHANGELOG parsing
    }

    return {
      description: npmData.description || "",
      homepage: npmData.homepage || "",
      repository: npmData.repository?.url || "",
      releaseNotes: releaseNotes || "No release notes found on GitHub.",
      changelogUrl,
      source,
    };
  },
});

```

Codefragment 5.2: fetchNpmInfoTool: npm-metadata en GitHub release notes met CHANGELOG fallback

Als de npm-tool geent nuttige release notes vindt, volgt de `webChangeLogSearchTool` (Listing 5.3). Deze tool voert een gerichte webzoekopdracht uit naar changelogs via Tavily, Brave Search of DuckDuckGo, met domeinpriorisering voor officiële package-sites (bijvoorbeeld `next.js.org`, `angular.dev`). De tool prioriteert resulta-

ten van officiële documentatie boven aggregators en levert een lijst van relevante titels, URL's en snippets.

```
export const webChangelogSearchTool = createTool({
  id: "web-changelog-search",
  inputSchema: z.object({
    packageName: z.string(),
    fromVersion: z.string(),
    toVersion: z.string(),
  }),
  execute: async ({ packageName, fromVersion, toVersion }) => {
    const query = buildQuery(packageName, fromVersion, toVersion);

    if (process.env.TAVILY_API_KEY) {
      return await searchWithTavily(query, packageName, toVersion);
    }
    return await searchWithDuckDuckGo(query); // fallback
  },
});
```

Codefragment 5.3: webChangelogSearchTool: webzoekopdracht naar changelogs en migratieguides

De Mastra-agent wordt vanuit de Next.js-backend aangeroepen via een API-route die de dependency-gegevens doorstuurt en de gestructureerde output opslaat in Supabase. Door de combinatie van deze twee tools en de expliciete instructies levert de agent betrouwbare, actionable samenvattingen die ontwikkelaars tijd besparen bij het beoordelen van updates.

5.3. Implementatie van de n8n-workflow

Als tweede AI-integratie wordt een n8n-workflow opgezet die dezelfde kernfunctie vervult als de Mastra-agent, maar dan in een low-code, visueel georkestreerde omgeving. De workflow ontvangt een HTTP-aanvraag vanuit de webapplicatie, verrijkt de aangeleverde dependency-informatie met data uit npm, Github en een webzoekdienst (Tavily)¹, en laat vervolgens een AI-model (Gemini 2.5 flash) een samenvatting genereren van de relevante wijzigingen. Het resultaat wordt teruggestuurd naar de applicatie via de webhook-respons en opgeslagen in de databank.

De workflow bestaat uit een keten van acht nodes (Figuur 5.3):

- **Node 1: Webhook.** Deze node fungeert als ingangspunt voor de workflow en wordt getriggerd zodra de Next.js-backend een HTTP-aanvraag naar de n8n-URL stuurt. De node is geconfigureerd om te antwoorden via een aparte Res-

¹Tavily is een API en infrastructuurlaag die AI-agents en LLM-toepassingen real-time toegang geeft tot webzoekopdrachten, webextractie en crawling.<https://www.tavily.com/>

pond to Webhook-node, zodat alle tussenliggende stappen eerst afgewerkt worden.

- **Node 2: npm-metadata (Code).** In deze code-node wordt de JSON-body van de inkomende request uitgelezen (`packageName`, `fromVersion`, `toVersion`) en wordt via `this.helpers.httpRequest` een call gedaan naar `https://registry.npmjs.org/{packageName}`. De node reconstrueert de repository-URL uit de npm-metadata en geeft een nieuwe JSON-object door met omschrijving, homepage en repository-informatie.
- **Node 3: Github-node (Code).** Deze node ontvangt de verrijkte npm-data, extraheert `owner/repo` uit de repository-URL en probeert via de Github API eerst release notes op te halen voor de specifieke tag (`v{toVersion}` of `toVersion`). Als die niet gevonden wordt, heelt de node de laatste releases op en concateneert de body-teksten tot een `releaseNotes`-veld. Als er ook geen releases zijn, wordt er nog geprobeerd een `CHANGELOG.md`, `CHANGES.md` of `HISTORY.md` rechtstreeks uit de repository te lezen. De node voegt `releaseNotes`, `changelogUrl` en `source` toe aan de JSON.
- **Node 4: HTTP Request (Tavily).** Via een HTTP-request-node wordt de Tavily API aangeroepen om webresultaten op te halen die extra context kunnen geven over de dependency of de update (bijvoorbeeld blogposts of documentatie rond de nieuwe versie).
- **Node 5: Merge-node (Code).** Deze node voegt de Github-data en Tavily-resultaten samen. De webresultaten worden omgezet naar een tekstveld `webSnippets` (titel, URL en inhoud per resultaat), dat samen met de eerder verzamelde metadata en release notes wordt doorgegeven aan de AI-node.
- **Node 6: AI-agent (Gemini).** Een AI-node, geconfigureerd met het model `gemini-2.5-flash`, ontvangt `releaseNotes`, `webSnippets` en versie-informatie als input. Met behulp van een system prompt en user prompt genereert het model een gestructureerde samenvatting van de update (bijvoorbeeld features, breaking changes, security-aspecten en migratieadvies).
- **Node 7: Code-node (post-processing).** Deze node extraheert de relevante output uit de AI-respons (bijvoorbeeld `output` of `summary`) en zet die om naar een eenvoudige JSON-object met één veld `summary`, zodat de webhook-respons helder en compact blijft.
- **Node 8: Respond to Webhook.** De laatste node stuurt de `summary` terug als HTTP-respons naar de oorspronkelijke aanroeper. Aan de kant van de webapplicatie kan deze samenvatting vervolgens rechtstreeks worden opgeslagen in de databank.

De Next.js-backend roept de n8n-workflow aan via een dedicated route (vereenvoudigd weergegeven in Listing 5.4). Deze route valideert eerst de gebruiker met Supabase, leest de dependencygegevens uit de inkomende request (dependencyId, name, currentVersion, latestVersion) en stuurt die als JSON-payload door naar de n8n-webhook. De samenvatting die n8n terugstuurt wordt daarna opgeslagen in het veld ai_summary van de desbetreffende dependency.

```

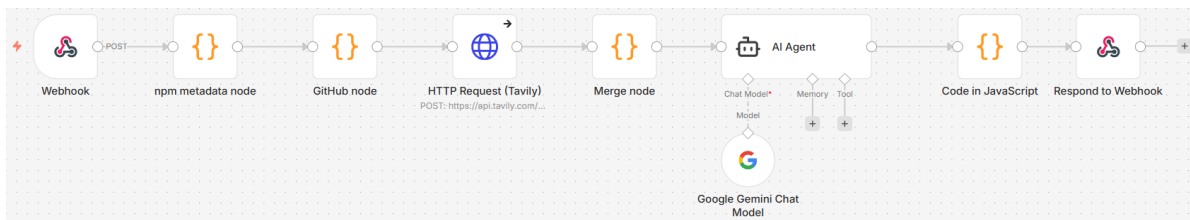
1 export async function POST(request: Request) {
2   const supabase = await createClient();
3   const { data: { user } } = await supabase.auth.getUser();
4   if (!user) {
5     return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
6   }
7
8   const { dependencyId, name, currentVersion, latestVersion } =
9     await request.json();
10
11   const n8nRes = await fetch(
12     process.env.N8N_WEBHOOK_URL ??
13       "http://localhost:5678/webhook-test/analyze-dependency",
14     {
15       method: "POST",
16       headers: { "Content-Type": "application/json" },
17       body: JSON.stringify({
18         packageName: name,
19         fromVersion: currentVersion,
20         toVersion: latestVersion,
21       }),
22     });
23
24   const n8nData = await n8nRes.json();
25   const summary = n8nData.summary;
26
27   await supabase
28     .from("dependencies")
29     .update({ ai_summary: summary })
30     .eq("id", dependencyId);
31
32   return NextResponse.json({ summary });
33 }

```

Codefragment 5.4: Next.js-route die de n8n-workflow aanroept en de samenvatting opslaat

Figuur 5.3 toont de volledige n8n-workflow zoals geïmplementeerd in deze bachelorproef, met de opeenvolgende nodes van inkomende webhook tot AI-gegenereerde

samenvatting.



Figuur 5.3: Overzicht van de n8n-workflow die metadata, release notes, webcontext en AI-samenvatting combineert

5.4. Implementatie van de OpenAI-agent-builder

5.5. Integratie met de webapplicatie

6

Resultaten en evaluatie

7

Conclusie

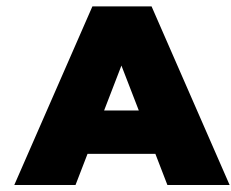
Op basis van de probleemstelling en onderzoeksdoelstelling wordt verwacht dat het proof-of-concept in staat zal zijn om dependency-updates automatisch te detecteren, te analyseren en in een compacte, begrijpelijke vorm te presenteren. De AI-agent zal changelogs en release notes samenvatten, het type wijziging (bijvoorbeeld beveiligingsfix, bugfix of nieuwe functionaliteit) aanduiden en een indicatie geven van de vermoedelijke impact op het project.

Verder wordt verwacht dat de werklast daalt doordat een deel van de beslissingen rond relatief veilige, backward compatible updates (zoals patch- of beperkte minor-releases) eenvoudiger of gedeeltelijk geautomatiseerd kan worden, terwijl potentieel risicovolle major-updates expliciet voor manuele review gemarkeerd blijven. Indien er voldoende meetdata beschikbaar komt (bijvoorbeeld logbestanden of tijdsregistraties), kan een eerste, kwantitatieve inschatting gemaakt worden van de tijdswinst in vergelijking met de huidige, hoofdzakelijk manuele aanpak.

De doelgroep, haalt hieruit meerwaarde doordat zij sneller prioriteiten kunnen bepalen, minder tijd verliezen aan het doorpluizen van changelogs en release notes en een beter overzicht krijgen van de actuele staat van dependencies en de bijhorende technische schuld. Daarnaast ontstaat een herhaalbaar proces dat later kan opgeschaald worden naar andere teams of projecten binnen de organisatie.

Verwacht wordt dat de evaluatie van de gekozen AI-agent- en/of workflowaanpak zal tonen dat geen enkele oplossing op alle criteria de beste is, maar dat duidelijke verschillen zichtbaar worden op het vlak van integratiemogelijkheden, gebruiksgemak, onderhoudbaarheid en performantie. Op basis van deze inzichten kan een onderbouwde aanbeveling worden geformuleerd voor een architectuur en toolkeuze die het best aansluit bij de context van Springbok, en kan geconcludeerd worden dat een AI-gestuurde laag een zinvolle stap is in het beperken van technische schuld rond dependencies. Indien de uiteindelijke resultaten afwijken van deze verwachtingen, biedt dit aanleiding om te onderzoeken welke aannames niet

bleken te kloppen en welke randvoorwaarden essentieel zijn voor succesvol AI-ondersteund dependencybeheer.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Softwareprojecten binnen bedrijven en organisaties maken gebruik van talrijke externe bibliotheken, frameworks en tools, de zogenaamde *dependencies*. Deze dependencies worden voortdurend bijgewerkt om nieuwe functionaliteiten, verbeterde prestaties en vooral beveiligingspatches aan te bieden. (Pashchenko et al., 2018) In de praktijk blijkt het voor ontwikkelteams echter moeilijk om alle updates systematisch op te volgen en grondig te evalueren, zeker wanneer een bedrijf meerdere projecten en repositories beheert.

A.1.1. Probleemstelling

Het manueel opvolgen van dependency-updates vormt een groot probleem in softwareontwikkeling. Ontwikkelaars moeten regelmatig changelogs doorzoeken, compatibiliteit controleren en inschatten welke updates veilig kunnen worden doorgevoerd, wat in de praktijk veel tijd vraagt en vaak wordt uitgesteld. (Behutiye et al., 2024; Pashchenko et al., 2018) Onderzoek toont aan dat veel softwareprojecten hun dependencies niet systematisch bijhouden, waardoor een aanzienlijk deel verouderd raakt. Pashchenko et al. (Pashchenko et al., 2018) stellen bijvoorbeeld dat een belangrijk percentage kwetsbare dependencies relatief eenvoudig opgelost kan worden door een update, maar dat dit in de praktijk niet gebeurt wegens tijdsdruk en gebrek aan overzicht. Wanneer het updateproces niet systematisch gebeurt, stapelen onderhoudstaken zich op, waardoor het steeds moeilijker en tijdrovender wordt om de software up-to-date te houden. Dit fenomeen staat bekend als

technische schuld (technical debt): de achterstand die ontstaat doordat noodzakelijke verbeteringen of updates te lang worden uitgesteld, wat leidt tot hogere onderhoudskosten en complexiteit op lange termijn.(Behutiye et al., 2024; Ruiz et al., 2024) Empirisch onderzoek bevestigt dat technische schuld binnen bedrijven reële negatieve gevolgen heeft voor productiviteit en softwarekwaliteit.(Besker, 2020) Om deze last te verlichten, zetten veel organisaties al geautomatiseerde dependencybots in, zoals Dependabot¹ of Renovate.² Deze tools maken wel automatisch pull requests aan zodra er nieuwe versies beschikbaar zijn,(Erlenhov et al., 2022) maar geven doorgaans weinig context over de inhoud en impact van de wijziging. Ontwikkelaars verliezen daardoor nog steeds tijd aan het interpreteren van changelogs, het inschatten van risico's en het beslissen of een update al dan niet moet worden doorgevoerd.(Erlenhov et al., 2022) Deze bachelorproef richt zich daarom op het ontwikkelen van een *aparte webapplicatie* (een dashboard) die integreert met GitHub-repositories van het bedrijf. Per project houdt deze applicatie de gebruikte dependencies bij, controleert via publieke registries (zoals npm,³) of er nieuwe versies beschikbaar zijn en verzamelt de relevante changelogs en release notes. Een AI-agent, die in dit onderzoek *from scratch* wordt ontworpen op basis van bestaande AI-modellen maar zonder voorafgebouwd agentframework, analyseert deze informatie, genereert begrijpelijke samenvattingen en beoordeelt in welke mate een update cruciaal is (bijvoorbeeld om veiligheidsredenen) of eerder optioneel. De AI-agent geeft het kernteam via het dashboard inzicht in vragen zoals: welke dependencies zijn verouderd, wat is er precies veranderd in de nieuwe versie, zijn er bekende beveiligingsrisico's als niet wordt geüpdatet, en lijkt de update technisch laag- of hoogrisicovol. De webapplicatie leeft dus als een losstaand platform naast GitHub, maar gebruikt GitHub uitsluitend als bron voor project- en dependency-informatie en, eventueel, als kanaal om later pull requests of issues te creëren. Op die manier kan het updateproces slimmer, sneller en inzichtelijker worden gemaakt.

A.1.2. Onderzoeksvraag

Om het probleem van handmatig dependencybeheer en beperkte context bij updates op te lossen, wordt de volgende hoofdonderzoeksvraag geformuleerd:

Hoe kan een AI-gedreven agent worden ingezet om dependency-updates uit bestaande tools voor automatisch dependencybeheer te analyseren en te beheren, zodat het kernteam dat verantwoordelijk is voor platform-

¹Dependabot is de ingebouwde GitHub-tool die kwetsbare of verouderde dependencies detecteert en automatisch update-pull-requests aanmaakt.<https://github.com/dependabot>

²Renovate is een open-source tool die automatisch update-pull-requests voor dependencies aanmaakt.<https://docs.renovatebot.com>

³npm is het standaard pakketbeheerplatform voor het JavaScript-ecosysteem.<https://www.npmjs.com>

en dependencybeheer efficiënter en met minder handmatig werk dependency-updates kan verwerken?

Hieruit vloeien de volgende deelvragen voort.

Deelvragen voor het beter begrijpen van het probleem

1. Wat zijn de belangrijkste uitdagingen en risico's bij het huidige, overwegend manuele dependency management voor het kernteam dat verantwoordelijk is voor platform- en dependencybeheer?
2. Welke bestaande tools en oplossingen voor automatisch dependencybeheer worden vandaag gebruikt, en welke sterke punten en beperkingen hebben deze oplossingen in de context van dit bedrijf?
3. Hoe leidt ophopende technische schuld op het vlak van dependency-updates ertoe dat er steeds meer tijd en inspanning nodig zijn voor het controleren en bijwerken van dependencies binnen het bedrijf?

Deelvragen richting de oplossing

4. Hoe kunnen AI-agents en workflow automatiseringsplatformen worden ingezet om informatie uit changelogs, release notes en dependency-pull-requests automatisch te interpreteren en in begrijpelijke vorm te communiceren naar het verantwoordelijke kernteam?
5. Welke functionele en niet-functionele vereisten moet een platform voor AI-gestuurd dependencybeheer vervullen, bijvoorbeeld op het vlak van integratie met versiebeheersystemen en registries, uitbreidbaarheid, beheer van regels en policies, auditlogging en performantie?
6. In welke mate voldoen geselecteerde AI-agent- of workflowplatformen aan deze vereisten, en hoe goed kunnen zij geïntegreerd worden in een proof-of-concept voor dependency management in de context van het bedrijf?

A.1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is om een proof-of-concept te ontwikkelen van een *aparte webapplicatie* (dashboard) die automatisch dependency-updates detecteert, analyseert en communiceert via een geïntegreerde, from-scratch ontworpen AI-agent, en om na te gaan in welke mate geselecteerde AI-agent- of workflowplatformen deze oplossing kunnen ondersteunen en voldoen aan de vereisten van het bedrijf voor geautomatiseerd dependencybeheer. Eerder onderzoek toont aan dat AI-technieken succesvol kunnen worden ingezet binnen onderhoudstaken zoals changelog-analyse en code review, wat aantoont dat deze toepassing haalbaar en relevant is. (Behutiye et al., [2024](#); Joshi, [2025](#); Kim et al., [2024](#); Zhu et al., [2025](#))

Concreet zal de oplossing:

- per project de gebruikte dependencies bijhouden;
- automatisch nagaan of er nieuwe versies beschikbaar zijn via publieke registers (zoals npm);
- changelogs en release notes analyseren met behulp van een AI-agent die samenvat wat er gewijzigd is en welke risico's of voordelen een update inhoudt;
- de verantwoordelijke developers via een dashboard informeren over de aard en impact van de updates, inclusief een korte, begrijpelijke samenvatting en een indicatie of de update cruciaal, risicovol of veilig lijkt;
- regels ondersteunen waardoor de developers gemakkelijker kan beslissen of en wanneer een dependency geüpdatet wordt, zonder dat deze beslissingen automatisch in GitHub worden afgedwongen.

De concrete technologiekeuzes worden in de verdere uitwerking van de bachelorproef gemotiveerd op basis van de requirements en de context van het bedrijf. Het onderzoek resulteert in:

- een prototype (demo) dat de werking van het systeem aantoont binnen een of enkele projecten van het bedrijf;
- een evaluatie van de toegevoegde waarde van AI bij dependency management voor de specifieke developer;
- een aanbeveling welk type platform, en eventueel welke concrete technologie, het meest geschikt is voor verdere toepassing binnen het bedrijf.

A.2. Literatuurstudie

A.2.1. Dependency management en kwetsbare dependencies

Moderne softwareprojecten steunen sterk op open-source dependencies, waardoor kwetsbaarheden in externe bibliotheken rechtstreeks een impact kunnen hebben op de veiligheid en betrouwbaarheid van toepassingen. (Pashchenko et al., 2018) Pashchenko et al. tonen aan dat een groot deel van de aangetroffen kwetsbare dependencies in de praktijk relatief eenvoudig verholpen kan worden door te upgraden naar een veiligere versie, maar dat ontwikkelteams deze updates vaak niet consequent doorvoeren. (Pashchenko et al., 2018)

In hun proefondervindelijk onderzoek onderscheiden Pashchenko et al. tussen daadwerkelijk gedeployde kwetsbare dependencies en libraries die enkel tijdens ontwikkeling of testing gebruikt worden, en stellen zij vast dat het merendeel van de reële kwetsbare dependencies door de eigen ontwikkelaars of directe onderhouders kan worden opgelost. (Pashchenko et al., 2018) Dit onderstreept het belang van systematisch dependencybeheer en duidelijke inzichten in welke dependencies effectief risico vormen in productie omgevingen. (Pashchenko et al., 2018)

A.2.2. Technische schuld en onderhoudslast

Het uitstellen van dependency-updates draagt bij aan de opbouw van technische schuld, waardoor onderhoud en toekomstige wijzigingen steeds meer inspanning vereisen.(Behutiye et al., 2024; Ruiz et al., 2024) Behutiye et al. analyseren de rol van technische schuld in agile omgevingen en beschrijven hoe keuzes die op korte termijn tijdswinst opleveren, op langere termijn leiden tot hogere kosten en complexere wijzigingen.(Behutiye et al., 2024)

Ruiz et al. bespreken in hun tertiaire studie dat technische-schuldmanagement de voorbije jaren een groeiend onderzoeksdomein is geworden, waarbij organisaties worstelen met het in kaart brengen, prioriteren en terugdringen van opgebouwde schuld.(Ruiz et al., 2024) Empirisch werk van Chalmers University of Technology bevestigt dat technische schuld niet louter een theoretisch concept is, maar in industriële context concreet voelbaar is in de vorm van verminderde productiviteit, hogere foutkans en vertragingen in softwarelevering.(Besker, 2020)

A.2.3. Dependencybots en hun beperkingen

Om de handmatige last rond dependencybeheer te verlagen, zetten veel projecten dependency management bots in, zoals Dependabot en Renovate.(Erlenhov et al., 2022) Erlenhov et al. voeren een kwantitatieve en kwalitatieve studie uit naar dependencybots in open-source systemen en stellen vast dat slechts een beperkt aantal van de geanalyseerde geautomatiseerde tools voldoet aan de criteria van volwaardige “Devbots”, die autonoom bijdragen aan de codebasis.(Erlenhov et al., 2022)

Hun resultaten tonen dat projecten vaak experimenteren met meerdere bots en regelmatig wisselen, onder meer door problemen met ruis (te veel pull requests), beperkte configureerbaarheid en moeilijkheden om de impact van voorgestelde updates goed te begrijpen.(Erlenhov et al., 2022) Hoewel dependencybots het detecteren en aanmaken van update-pull-requests automatiseren, blijft de inhoudelijke beoordeling van changelogs, compatibiliteit en risico's grotendeels bij ontwikkelaars liggen, wat aansluit bij de probleemstelling van deze bachelorproef.(Erlenhov et al., 2022; Pashchenko et al., 2018)

A.2.4. AI, agents en workflow automatisering

Recente literatuur verkent hoe AI-agents en multi-agentframeworks kunnen worden ingezet om complexe taken in softwareontwikkeling en data-intensieve workflows te coördineren.(Joshi, 2025; Kim et al., 2024; Zhu et al., 2025) Joshi biedt een overzicht van autonome systemen en collaboratieve AI-agentframeworks en benadrukt dat zulke agents vooral nuttig zijn in scenario's met veel herhalende beslissingen op basis van tekstuele en gestructureerde input, zoals logs, notificaties en rapporten.(Joshi, 2025)

Kim et al. introduceren met Bel Esprit een multi-agentframework voor het opbou-

wen van AI-model-pijplijnen, waarbij verschillende gespecialiseerde agents samenwerken om complexere taken modulair af te handelen.(Kim et al., 2024) Zhu et al. bestuderen in OAgents empirisch welke ontwerpkeuzes leiden tot effectieve agents, en bespreken onder meer het belang van duidelijke taakafbakening, robuuste toolintegratie en feedbackloops voor betrouwbare beslissingsondersteuning.(Zhu et al., 2025) Deze inzichten zijn relevant voor het ontwerp van een AI-agent die dependency-updates interpreteert, samenvat en beslissingsvoorstellen formuleert voor een onderhoudsteam.

Naast generieke agentframeworks beschrijft Wali et al. hoe workflow-automatisering met n8n⁴ operationele efficiëntie kan verhogen door repetitieve, gegevensgedreven processen te coördineren over verschillende systemen heen.(Wali et al., 2025) Hoewel hun casus zich richt op een financiële context, illustreert de studie dat low-code workflowtools geschikt zijn om integraties met externe APIs, databronnen en notificatiekanalen te realiseren, wat ook essentieel is voor een geautomatiseerde dependency-updatepipeline.(Wali et al., 2025)

A.2.5. Onderzoeksniche: AI-ondersteund dependencybeheer

Samengenomen schetst de literatuur een duidelijk beeld: kwetsbare of verouderde dependencies vormen een reëel risico, maar kunnen vaak verholpen worden door updates die vandaag onvoldoende systematisch worden uitgevoerd.(Behutiye et al., 2024; Pashchenko et al., 2018) Tegelijk tonen studies naar technische schuld dat het structureel uitstellen van onderhoud, waaronder dependency updates, leidt tot een groeiende onderhoudslast en achterstand.(Behutiye et al., 2024; Besker, 2020; Ruiz et al., 2024)

Onderzoek naar dependencybots maakt duidelijk dat huidige oplossingen weliswaar updates detecteren en pull requests genereren, maar ontwikkelaars nog steeds belasten met het interpreteren van changelogs en het beoordelen van risico's en prioriteiten.(Erlenhov et al., 2022) De recente vooruitgang in AI-agents en workflow-automatisering suggereert dat een volgende stap mogelijk is: een systeem waarin een AI-agent niet alleen updates signaleert, maar ook de inhoud van release notes en changelogs analyseert, samenvat en contextafhankelijk advies geeft aan een klein kernteam dat verantwoordelijk is voor dependencybeheer.(Joshi, 2025; Kim et al., 2024; Wali et al., 2025; Zhu et al., 2025)

Deze bachelorproef positioneert zich precies in deze niche door een AI-gestuurde laag bovenop bestaande dependencybots te onderzoeken en te prototypen, met de expliciete focus op het verlagen van de manuele beoordelingslast en het beheersbaar houden van technische schuld rond dependencies in een bedrijfscontext.

⁴n8n is een open-source tool om workflows en API's visueel te automatiseren.<https://n8n.io>

A.3. Methodologie

Dit onderzoek volgt de opzet van een vergelijkende studie met proof-of-concept, aangevuld met literatuuronderzoek en een requirements-analyse binnen een concrete bedrijfscontext. Het doel is om enerzijds het probleem en de oplossingsruimte rond dependencybeheer in kaart te brengen, en anderzijds een apart dashboard-prototype te bouwen en technisch te evalueren binnen een realistische ontwikkelomgeving.

A.3.1. Fase 1 - Verdiepende literatuurstudie en probleemverkenning

In de eerste fase van de bachelorproef wordt de probleemcontext verder uitgediept aan de hand van een systematische literatuurstudie rond dependency management, kwetsbare dependencies, technische schuld en bestaande dependencybots. Daarnaast wordt vakliteratuur over AI-agents en workflow-automatisering bestudeerd om na te gaan welke concepten en benaderingen relevant zijn voor een AI-gestuurd dependency-dashboard.

Tijdens deze fase worden ook één of enkele representatieve projecten van het bedrijf geselecteerd en geanalyseerd (projectstructuur, gebruikte technologieën, dependency-ecosystemen, huidige processen rond updates). De belangrijkste deliverables zijn:

- een uitgewerkte en geactualiseerde literatuurstudie;
- een scherpere probleemomschrijving toegespitst op de gekozen bedrijfscontext en projecten;
- een eerste set geïnformeerde hypothesen over de verwachte impact van een AI-gestuurd dependency-dashboard.

A.3.2. Fase 2 - Requirements-analyse en architectuurontwerp

In de tweede fase wordt eerst een gerichte requirements-analyse uitgevoerd in samenwerking met de co-promotor via interviews en workshops worden functionele en niet-functionele eisen expliciet gemaakt, zoals:

- functionele eisen: integratie met GitHub, beheer van projecten en dependencies in het dashboard, detectie van nieuwe versies via publieke registries, analyse en samenvatting van changelogs en release notes, aanduiding van kriticiiteit (security, bugfix, feature), prioritering van updates;
- niet-functionele eisen: schaalbaarheid, betrouwbaarheid, traceerbaarheid (logging en audittrail), gebruiksvriendelijkheid en onderhoudbaarheid.

Op basis van deze requirements wordt vervolgens een systeemarchitectuur ontworpen voor de webapplicatie en de from-scratch ontworpen AI-agent. De architectuur beschrijft onder meer de globale componenten (frontend, backend, databank, integraties met GitHub en registries), de verantwoordelijkheden per component, de datastromen (bijvoorbeeld van dependency-informatie naar AI-analyse en

terug naar het dashboard) en de plaats van eventueel ondersteunende workflow- of agentplatformen. Deliverables van fase 2 zijn:

- een requirementsdocument met duidelijke prioritering;
- een architectuurbeschrijving (diagrammen, componentbeschrijvingen, data-modellen);
- een lijst van geselecteerde technologieën en tools, gemotiveerd vanuit de requirements.

A.3.3. Fase 3 - Implementatie van het proof-of-concept

In fase 3 wordt de ontworpen architectuur omgezet in een werkend proof-of-concept. De kern bestaat uit een aparte webapplicatie (dashboard) waarin per project de gebruikte dependencies worden bijgehouden en verrijkt met informatie uit publieke registries zoals npm. De applicatie werkt vanaf het begin met één of meerdere reële GitHub-repositories van het bedrijf, zodat de proof-of-concept onmiddellijk getest wordt in een realistische context. Dependency-informatie wordt automatisch uit deze repositories gehaald (bijvoorbeeld via repositoryconfiguratiebestanden) en opgeslagen in een centrale databank.

Parallel wordt de AI-agentlaag geïmplementeerd in de backend. In deze fase worden drie concrete AI-agentoplossingen opgezet en geïntegreerd in hetzelfde dashboard: een agent op basis van OpenAI Agent Builder, een workflow met n8n en een agentconfiguratie met Flowise. Elk van deze varianten ontvangt changelogs en release-note-informatie, genereert beknopte samenvattingen en beoordeelt de aard en vermoedelijke impact van updates (bijvoorbeeld beveiligingsfix, bugfix, nieuwe functionaliteit, potentieel brekende wijziging). De frontend visualiseert de status van dependencies, aanbevolen acties en de door de verschillende AI-varianten gegenereerde toelichting. Deliverables van fase 3 zijn:

- een werkend prototype van het dashboard met integratie naar GitHub en één dependency-registries;
- drie geïmplementeerde AI-agentvarianten (OpenAI Agent Builder, n8n, Flowise) die changelogs en release notes verwerken en samenvatten;
- technische documentatie over de implementatie (architectuur, API's, datamodellen en integratiepunten voor de agents).

A.3.4. Fase 4 - Vergelijkende onderbouwing van architectuur en AI-aanpak

De vierde fase richt zich op het onderbouwen en verfijnen van de gemaakte ontwerpkeuzes aan de hand van een gerichte vergelijking van de drie uitgewerkte AI-agentoplossingen en eventuele architecturale varianten. Op basis van de requi-

rements uit fase 2 wordt een set *eenvoudig meetbare* evaluatiecriteria opgesteld, zoals:

- gemiddelde verwerkingstijd per update (doorlooptijd van input tot samenvatting);
- technische inspanning voor opzet en onderhoud (bijvoorbeeld aantal configuratiestappen, hoeveelheid code);
- kwaliteit van samenvattingen gemeten via een eenvoudige beoordelingsschaal door de co-promotor (bijvoorbeeld 1–5 voor duidelijkheid en relevantie);
- fout- of faalratio (bijvoorbeeld aantal mislukte runs of onvolledige resultaten per tien updates).

Aan de hand van een aantal representatieve scenario's (bijvoorbeeld updates met alleen kleine bugfixes, updates met beveiligingspatches, updates met potentieel brekende wijzigingen) worden OpenAI Agent Builder, n8n en Flowise onder dezelfde omstandigheden getest en met elkaar vergeleken. De focus ligt hierbij niet op een brede marktanalyse, maar op het versterken of bijsturen van de gekozen architectuur en het bepalen welke aanpak het meest geschikt is als basis voor het verdere gebruik van het dashboard in deze specifieke use case. Deliverables van fase 4 zijn:

- experimentele beschrijvingen van de uitgeteste varianten;
- meetresultaten volgens de gedefinieerde criteria (doorlooptijd, inspanning, beoordelingsscores, foutpercentages);
- een gemotiveerde keuze of bevestiging van de uiteindelijke architectuur en AI-configuratie.

A.3.5. Fase 5 - Evaluatie en rapportering

In de laatste fase wordt het proof-of-concept geëvalueerd met de co-promotor binnen het bedrijf, aan de hand van een combinatie van demonstraties, gericht feedbackgesprekken en, waar mogelijk, beperkte praktijktesten realistische projecten. De evaluatie focust op bruikbaarheid van het dashboard, de gemeten vermindering van manuele analysetijd (bijvoorbeeld aantal geanalyseerde updates per uur vóór en na gebruik), de door de co-promotor toegekende beoordelingsscores voor duidelijkheid van de AI-samenvattingen en de mate waarin het systeem helpt bij de prioritering van updates.

De bevindingen uit deze evaluatie worden gecombineerd met de kwantitatieve resultaten van fase 4 om de onderzoeksvragen te beantwoorden en de onderzoekshypothesen te toetsen. Deliverables van fase 5 zijn:

- een synthese van de evaluatie met de co-promotor (inclusief gemeten tijdswinst, kwaliteitsbeoordelingen en verbetervoorstellen);
- de uitgewerkte bachelorproef met een geïntegreerde beschrijving van probleem, methode, resultaten en conclusies;
- aanbevelingen voor verdere ontwikkeling en mogelijke integratie van het systeem in de bredere ontwikkelprocessen van het bedrijf.

De globale fasering en tijdsplanning van het onderzoek wordt weergegeven in Figuur A.1.

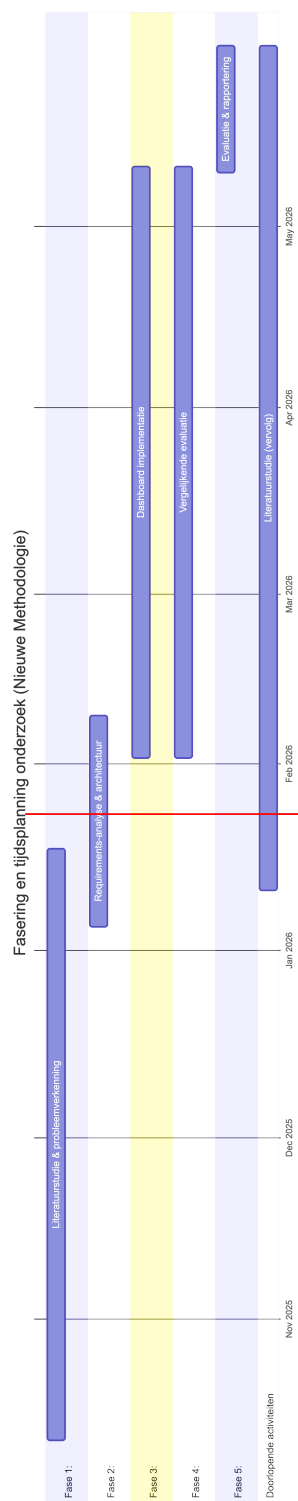
A.4. Verwacht resultaat en conclusie

Op basis van de probleemstelling en onderzoeksdoelstelling wordt verwacht dat het proof-of-concept in staat zal zijn om dependency-updates automatisch te detecteren, te analyseren en in een compacte, begrijpelijke vorm te presenteren. De AI-agent zal changelogs en release notes samenvatten, het type wijziging (bijvoorbeeld beveiligingsfix, bugfix of nieuwe functionaliteit) aanduiden en een indicatie geven van de vermoedelijke impact op het project.

Verder wordt verwacht dat de werklast daalt doordat een deel van de beslissingen rond relatief veilige, backward compatible updates (zoals patch- of beperkte minor-releases) eenvoudiger of gedeeltelijk geautomatiseerd kan worden, terwijl potentieel risicovolle major-updates expliciet voor manuele review gemarkeerd blijven. Indien er voldoende meetdata beschikbaar komt (bijvoorbeeld logbestanden of tijdsregistraties), kan een eerste, kwantitatieve inschatting gemaakt worden van de tijdswinst in vergelijking met de huidige, hoofdzakelijk manuele aanpak.

De doelgroep, haalt hieruit meerwaarde doordat zij sneller prioriteiten kunnen bepalen, minder tijd verliezen aan het doorpluizen van changelogs en release notes en een beter overzicht krijgen van de actuele staat van dependencies en de bijhorende technische schuld. Daarnaast ontstaat een herhaalbaar proces dat later kan opgeschaald worden naar andere teams of projecten binnen de organisatie.

Verwacht wordt dat de evaluatie van de gekozen AI-agent- en/of workflowaanpak zal tonen dat geen enkele oplossing op alle criteria de beste is, maar dat duidelijke verschillen zichtbaar worden op het vlak van integratiemogelijkheden, gebruiksgemak, onderhoudbaarheid en performantie. Op basis van deze inzichten kan een onderbouwde aanbeveling worden geformuleerd voor een architectuur en toolkeuze die het best aansluit bij de context van het bedrijf, en kan geconcludeerd worden dat een AI-gestuurde laag een zinvolle stap is in het beperken van technische schuld rond dependencies. Indien de uiteindelijke resultaten afwijken van deze verwachtingen, biedt dit aanleiding om te onderzoeken welke aannames niet bleken te kloppen en welke randvoorwaarden essentieel zijn voor succesvol AI-ondersteund dependencybeheer.



Figuur A.1: Fasering en tijdsplanning van het onderzoek

Bibliografie

- AI, M. (2025a). *Mastra: The Javascript framework for building AI agents* [Beschrijving van Mastra als JavaScript-framework voor AI-agents met workflows, memory en evals]. Geraadpleegd op 2 maart 2026, van <https://www.ycombinator.com/companies/mastra>
- AI, M. (2025b). *Observability Overview* [Documentatie over tracing en observability voor Mastra-agents en workflows]. Geraadpleegd op 2 maart 2026, van <https://mastra.ai/docs/observability/overview>
- AI, M. (2026). *Mastra: The TypeScript AI Agent Framework* [Open-source framework voor het bouwen van AI-agents en workflows in TypeScript]. Geraadpleegd op 2 maart 2026, van <https://github.com/mastra-ai/mastra>
- Behutiye, W. N., Rodriguez, P., Oivo, M., & Tosun, A. (2024). Analyzing the Concept of Technical Debt in the Context of Agile Software Development: A Systematic Literature Review. *arXiv*. <https://arxiv.org/abs/2401.14882>
- Besker, T. (2020). *Technical Debt: An Empirical Investigation of Its Harmfulness and Management Strategies in Industry* [PhD thesis]. Chalmers University of Technology. <https://research.chalmers.se/en/publication/518318>
- Codecademy. (2025). *What is n8n: Build AI Workflows with n8n* [Handleiding voor het opzetten van AI-workflows in n8n met de visuele editor]. Geraadpleegd op 9 maart 2026, van <https://www.codecademy.com/article/build-ai-workflows-with-n8n>
- Erlenhov, L., De Oliveira Neto, F. G., & Leitner, P. (2022). Dependency Management Bots in Open-Source Systems—Prevalence and Adoption. *PeerJ Computer Science*, 8, e849. <https://doi.org/10.7717/peerj-cs.849>
- Hatchworks. (2025). *n8n Guide 2026: Features & Workflow Automation Deep Dive* [Diepgaande gids over n8n met nadruk op AI-workflows, triggers en integraties]. Geraadpleegd op 9 maart 2026, van <https://hatchworks.com/blog/ai-agents/n8n-guide/>
- Infralovers. (2025). *n8n Workflow Automation* [Introductie tot n8n met focus op open-source, integraties en typische gebruiksscenario's]. Geraadpleegd op 9 maart 2026, van <https://www.infralovers.com/blog/2025-05-09-n8n-workflow-automation/>
- Joshi, S. (2025). Review of Autonomous Systems and Collaborative AI Agent Frameworks. *International Journal of Science and Research Archive*, 14(02), 961–972. <https://doi.org/10.30574/ijrsra.2025.14.2.0439>

- Kim, Y., Abdelaziz, A. E., Castro Ferreira, T., Al-Badrashiny, M., & Sawaf, H. (2024). Bel Esprit: Multi-Agent Framework for Building AI Model Pipelines. *arXiv*. <https://doi.org/10.48550/arXiv.2412.14684>
- n8n GmbH. (2025a). *n8n* [Overzichtsartikel over n8n als low-code, visuele workflow-automatiseringstool]. Geraadpleegd op 9 maart 2026, van <https://en.wikipedia.org/wiki/N8n>
- n8n GmbH. (2025b). *n8n Documentation: About n8n* [Officiële documentatie over n8n als workflow-automatiseringsplatform]. Geraadpleegd op 9 maart 2026, van <https://docs.n8n.io>
- Nwachukwu, U. (2025). *Creating AI Agents with Mastra and TypeScript* [Artikel dat uitlegt hoe Mastra-agents en tools worden gedefinieerd in TypeScript]. Geraadpleegd op 2 maart 2026, van <https://dev.to/ucodes/creating-ai-agents-with-mastra-and-typescript-4d6o>
- Pashchenko, I., Plate, H., Ponta, S. E., Sabetta, A., & Massacci, F. (2018). Vulnerable Open Source Dependencies: Counting Those That Matter. *Proceedings of the 12th International Symposium on Empirical Software Engineering and Measurement (ESEM)*.
- Proser, Z. (2025). *n8n: The Workflow Automation Tool for the AI Age* [Artikel dat de architectuur, event-gedreven werking en AI-toepassingen van n8n beschrijft]. Geraadpleegd op 9 maart 2026, van <https://workos.com/blog/n8n-the-workflow-automation-tool-for-the-ai-age>
- Ruiz, J. A., Aguilar, R. A., Garcilazo, J. F., & Aguilera, A. A. (2024). A Tertiary Study on Technical Debt Management over the last lustrum. *International Journal of Combinatorial Optimization Problems and Informatics*, 15(5), 181–192. <https://doi.org/10.61467/2007.1558.2024.v15i5.577>
- The AI Builders. (2025). *Mastra: Build Powerful AI Agents with Typescript* [Tutorial met voorbeeldagents, tools en workflows in Mastra]. Geraadpleegd op 2 maart 2026, van https://tutorial.theaibuilders.dev/tutorials/Frameworks/mastra_tutorial
- Wali, M., Nasir, N., & Iqbal, T. (2025). Implementing Workflow Automation with n8n to Enhance Operational Efficiency and Performance in the Sharia Cooperative of Bank Indonesia, Aceh Province. *Journal Digital Technology Trend*, 4(1), 36–47. <https://doi.org/10.56347/jdtt.v4i1.341>
- Zhu, H., Qin, T., Zhu, K., Huang, H., Guan, Y., Xia, J., Yao, Y., Li, H., Wang, N., Liu, P., Peng, T., Gui, X., Li, X., Liu, Y., Jiang, Y. E., Wang, J., Zhang, C., Tang, X., Zhang, G., ... Zhou, W. (2025). OAgents: An Empirical Study of Building Effective Agents. <https://arxiv.org/abs/2506.15741>